



Project Report: Serverless Circuit Breaker

Target Exam: AWS Certified Developer - Associate (DVA-C02) Goal Score: 900+

1. Architectural Overview

This project implemented a resilient, serverless payment processing system designed to handle downstream API failures gracefully.

- **Core Services:** API Gateway, AWS Lambda, Amazon DynamoDB, AWS SAM.
- **The Anti-Pattern Solved:** Preventing AWS Lambda from hanging synchronously while waiting for a broken 3rd-party API to respond, which prevents massive bill spikes and API Gateway timeouts.

2. The Circuit Breaker State Machine

The architecture relies on an external state store (DynamoDB) to track the health of the payment API across all concurrent Lambda invocations.

- **CLOSED:** The external API is healthy. Lambda processes requests normally.
- **OPEN ("Failing Fast"):** The failure threshold (e.g., 3 failures) is reached. Lambda instantly returns a `503 Service Unavailable` without attempting to call the external API. This saves compute time (milliseconds) and money.
- **HALF-OPEN:** After a cooling period (e.g., 30 seconds), one request is allowed through to test if the external API has recovered. If it succeeds, state returns to CLOSED. If it fails, state returns to OPEN.

3. High-Yield DVA-C02 Exam Observations

A. AWS Lambda Mechanics

- **Execution Environment Reuse (Cold Starts):** To optimize performance, database connections (e.g., `boto3.resource('dynamodb')`) must be initialized *outside* the `lambda_handler`. This allows warm execution environments to reuse the connection.
- **Statelessness & Scaling:** You cannot use a global Python variable to store the `FailureCount`. Because Lambda scales out horizontally by spinning up isolated environments, concurrent requests do not share memory. State *must* be externalized to DynamoDB or ElastiCache.
- **Synchronous vs. Asynchronous Retries (CRITICAL TRAP):** * API Gateway triggers Lambda **synchronously**. AWS *does not* automatically retry synchronous failures. The client (frontend) must implement Exponential Backoff.
 - If Lambda was triggered **asynchronously** (e.g., by S3 or SNS), AWS *would* automatically retry the execution twice before sending it to a Dead Letter Queue (DLQ).

B. Amazon DynamoDB

- **Cost Optimization (Billing Mode):** The project utilized On-Demand capacity (`PAY_PER_REQUEST`). The exam will test your ability to choose between Provisioned Capacity (predictable workloads) and On-Demand (spiky, unpredictable workloads or architectures that sit idle).

- **Concurrency and Race Conditions:** * *The Flaw:* Reading a value, adding 1 in code, and writing it back using `PutItem` is dangerous at scale. 100 concurrent Lambdas will overwrite each other.
 - *The Exam Solution:* You must use `UpdateItem` with an **Atomic Counter** (`UpdateExpression="ADD FailureCount :inc"`) to safely increment values in highly concurrent serverless applications.
- **TTL (Time to Live):** The system uses `ExpirationTime` to automatically delete old circuit breaker records from DynamoDB without incurring write costs, keeping the database clean.

C. Amazon API Gateway

- **Hard Timeouts:** API Gateway has an absolute, unchangeable maximum timeout of **29 seconds**. If a downstream API takes 35 seconds, API Gateway will drop the connection and return a `504 Gateway Timeout`. The Circuit Breaker prevents Lambda from hitting this limit.
- **Proxy Integration Payloads:** When using Lambda Proxy Integration, the HTTP body is passed to Lambda as a stringified JSON object. You must explicitly parse it (e.g., `json.loads(event.get('body'))`).

D. AWS SAM (Serverless Application Model)

- **Implicit vs. Explicit APIs:** By simply adding an `Events: Type: Api` property to the Lambda function in `template.yaml`, **SAM implicitly** creates the API Gateway resources behind the scenes. You only need explicit `AWS::Serverless::Api` resources for advanced configurations like Cognito Authorizers or CORS.

4. Final Verdict for Exam Prep

This architecture proves that just making services talk to each other isn't enough for the DVA-C02 exam. You are expected to design for failure. Understanding how to use external state (DynamoDB) to coordinate fast-failing logic across stateless compute (Lambda) is a hallmark of a senior AWS developer.